

[← Go Back](#)

Hardware

Due



Tutorials

Extended SPI Library Usage with the Arduino Due

[Due Motor Shield Example](#)[Arduino Due Keyboard Controller](#)[Mouse Controller](#)[Due Multiple Blinks Example](#)[Simple Waveform Generator with Arduino Due](#)[Keyboard Controller](#)[Upgrading Firmware of the Arduino Due Programming Port 16U2](#)[Home](#) / [Hardware](#) / [Due](#) /

Extended SPI Library Usage with the Arduino Due

ON THIS PAGE

Use

Extended SPI Library Usage with the Arduino Due

The SAM3X has advanced SPI capabilities. It is possible to use these extended methods, or the AVR-based ones.

Author • Arduino Last revision • 03/15/2023

These APIs should not be used in new projects. Use [SPISettings](#) with [SPI.beginTransaction\(\)](#) to configure SPI parameters.

On the Arduino Due, the SAM3X has advanced SPI capabilities. It is possible to use these extended methods, or the AVR-based ones.

The extended API can use pins 4, 10, and 52 for CS.

Use

You must specify each pin you wish to use as CS for the SPI devices.

It is possible for the Due to automatically handle the chip selection between multiple devices sharing the SPI bus. Each device may have also different attributes such as speed and

[Help](#)

If using multiple devices with different CS pins, you'll need to declare those pins in `setup()`. In the following example, there are two devices that share the SPI MISO, MOSI, and SCK pins. One device CS is attached to pin 4, the other to pin 10.

```
1 void setup(){
2   // initialize the bus
3   SPI.begin(4);
4   // initialize the bus
5   SPI.begin(10);
6 }
```

Once a pin has been declared as a CS pin, it's possible to change its default behaviors as well. For example, if the devices run at different clock speeds, the `setup()` may look like this:

```
1 void setup(){
2   // initialize the bus
3   SPI.begin(4);
4   // Set clock divider
5   SPI.setClockDivider(
6   // initialize the bus
7   SPI.begin(10);
8   // Set clock divider
9   SPI.setClockDivider(
10 }
```

A single byte transfer to a device on pin 4 could look like this :

```
1 void loop(){
2   byte response = SPI.t
```

In the above, "0xFF" is sent to the SPI device on pin 4 and the data coming from MISO is saved inside the variable response. The chip selection is handled automatically by the SPI controller, the transfer command implies the following:

Select device by setting pin 4 to LOW

Send 0xFF through the SPI bus and return the byte received

Deselect device by setting pin 4 to HIGH

It's possible to send more than one byte in a transaction by telling the the transfer command to not deselect the SPI device after the transfer :

```
1 void loop(){
2 //transfer 0x0F to the
3 SPI.transfer(10, 0xF0,
4 //transfer 0x00 to the
5 SPI.transfer(10, 0x00,
6 //transfer 0x00 to the
7 byte response1 = SPI.t
8 //transfer 0x00 to the
9 byte response2 = SPI.t
10 }
```

The parameter SPI_CONTINUE ensures that chip selection is keep active between transfers. On the last transfer SPI_CONTINUE is not specified as it's the last byte transferred.

See the individual reference pages for [setClockDivider\(\)](#), [setDataMode\(\)](#), [transfer\(\)](#),

when using the extended methods.

NB : once `SPI.begin()` is called, the declared pin will not be available as a general purpose I/O pin.

Suggest changes

The content on [docs.arduino.cc](#) is facilitated through a public [GitHub repository](#). If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino](#)
[Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

